

1. Justificación: BST vs ArrayList para búsquedas frecuentes Cuando se trabaja con grandes volúmenes de datos históricos, como es el caso de AncestryDigital con millones de registros, la elección de la estructura de datos tiene un impacto directo en el rendimiento del sistema.

1.1 Complejidad temporal de búsqueda: En un ArrayList, los elementos se almacenan de forma secuencial en memoria. Para encontrar un elemento con un ID específico se debe recorrer la lista desde el inicio hasta encontrar la coincidencia. En el peor caso (el elemento está al final o no existe) esto implica revisar los n elementos, lo que da una complejidad de $O(n)$. Si el ArrayList estuviera ordenado y se aplicara búsqueda binaria, se podría lograr $O(\log n)$, pero eso requeriría mantenerlo ordenado en cada inserción, lo cual es $O(n)$ por inserción. En un BST, cada nodo descarta aproximadamente la mitad del árbol en cada comparación. Esto equivale a dividir el espacio de búsqueda en dos en cada paso, lo que da una complejidad promedio de $O(\log n)$ tanto para búsqueda como para inserción y eliminación.

1.2 Ventaja del orden implícito in-orden: Una ventaja clave del BST es que mantiene los datos organizados de forma jerárquica sin necesidad de operaciones adicionales. El recorrido in-orden del árbol produce automáticamente los registros ordenados por su ID de menor a mayor, lo cual es esencial para los reportes de AncestryDigital. Con un ArrayList desordenado sería necesario aplicar un algoritmo de ordenamiento adicional $O(n \log n)$ cada vez que se genera el reporte.

1.3 Escalabilidad: Supongamos que AncestryDigital tiene 1 000 000 de registros. Una búsqueda en ArrayList revisaría hasta 1 000 000 nodos. El BST, en un árbol razonablemente balanceado, realizaría aproximadamente $\log_2(1\,000\,000) \approx 20$ comparaciones. Esa diferencia es muy notoria en un sistema con búsquedas frecuentes y concurrentes

2. Lógica de eliminación de nodos en el BST : La eliminación es la operación más compleja de un BST porque debe garantizar que la propiedad del árbol ($ID \text{ izquierdo} < ID \text{ raíz} < ID \text{ derecho}$) se mantenga después de retirar cualquier nodo. Se contemplan tres casos.

Caso 1 — Nodo hoja (sin hijos)

Es el caso más simple. El nodo a eliminar no tiene hijos izquierdo ni derecho, por lo que se puede retirar directamente retornando None al nodo padre. La propiedad BST se conserva trivialmente.

```
# CASO 1: Nodo hoja
if nodo.izquierdo is None and nodo.derecho is None:
    return None
```

Caso 2 — Nodo con un solo hijo

Si el nodo tiene únicamente un hijo (izquierdo o derecho), se reemplaza el nodo eliminado por ese hijo. El padre del nodo eliminado apuntará directamente al nieto, saltando el nodo que se elimina. La propiedad BST se conserva porque el subárbol del hijo ya cumplía la propiedad

```
# CASO 2a: Solo hijo derecho
if nodo.izquierdo is None:
    return nodo.derecho

# CASO 2b: Solo hijo izquierdo
if nodo.derecho is None:
    return nodo.izquierdo
```

Caso 3 — Nodo con dos hijos (el más delicado)

Cuando el nodo a eliminar tiene dos hijos no se puede simplemente desconectarlo porque habría que elegir cuál de los dos hijos lo reemplaza manteniendo el orden. La solución mas normal es usar el sucesor in-order: el nodo con el ID más pequeño dentro del subárbol derecho del nodo a eliminar. El sucesor in-order es el candidato perfecto porque:

- Es mayor que todos los nodos del subárbol izquierdo del nodo eliminado.
- Es menor que todos los demás nodos del subárbol derecho (porque es el mínimo de ese subárbol).
- Por lo tanto, ocupa exactamente la posición correcta para mantener la propiedad BST.

El procedimiento es:

- Encontrar el sucesor in-order: descender siempre por la izquierda desde el hijo derecho del nodo a eliminar hasta llegar a un nodo sin hijo izquierdo.
- Copiar el dato del sucesor al nodo que se quería eliminar (en lugar de moverlo físicamente).
- Eliminar recursivamente el sucesor de su posición original (que será Caso 1 o Caso 2, nunca Caso 3 de nuevo, ya que el sucesor tiene a lo sumo un hijo derecho).

```
# CASO 3: Dos hijos → sucesor in-order
sucesor = self._encontrar_minimo(nodo.derecho)
nodo.dato = sucesor.dato
nodo.derecho = self._eliminar_rec(nodo.derecho, sucesor.dato.id)

def _encontrar_minimo(self, nodo):
    """Retorna el nodo con el ID mínimo del subárbol."""
    while nodo.izquierdo is not None:
        nodo = nodo.izquierdo
    return nodo
```

3. Análisis de complejidad temporal

Para cada operación implementada se analiza la complejidad en tres escenarios: mejor caso, caso promedio y peor caso. El peor caso ocurre cuando el árbol está completamente degenerado (todos los nodos en una sola cadena), lo cual sucede si los datos se insertan en orden estrictamente creciente o decreciente.

3.1 Complejidad espacial

El árbol almacena n nodos, por lo que la complejidad espacial es $O(n)$. Las funciones recursivas utilizan la pila de llamadas del sistema, lo que añade $O(h)$ de espacio adicional (donde h es la altura del árbol), $O(\log n)$ en promedio y $O(n)$ en el peor caso.

3.2 Observación sobre el árbol de prueba

El árbol construido con los datos de prueba insertando en el orden 50, 30, 70, 20, 40, 60, 80 produce un árbol perfectamente balanceado de altura 3. Con 7 nodos y altura 3, $\log_2(7) \approx 2.8$, lo que confirma el comportamiento $O(\log n)$ esperado

4. Evidencias — Recorridos y operaciones

Estos son los datos “Quemados”.

```
arbol.insertar(RegistroHistorico(50, "Acta Fundacional Medellín", 1675, "Acta"))
arbol.insertar(RegistroHistorico(30, "Carta Gobernación Antioquia", 1813, "Carta"))
arbol.insertar(RegistroHistorico(70, "Escritura Hacienda El Retiro", 1890, "Escritura"))
arbol.insertar(RegistroHistorico(20, "Bautismo Gregorio Gutiérrez", 1801, "Registro Civil"))
arbol.insertar(RegistroHistorico(40, "Matrimonio Familia Ospina", 1856, "Registro Civil"))
arbol.insertar(RegistroHistorico(60, "Defunción Juan de Dios Uribe", 1900, "Registro Civil"))
arbol.insertar(RegistroHistorico(80, "Testamento Tomás Cipriano", 1862, "Testamento"))
```

4.1 Recorrido In-Orden

El recorrido in-orden recorre el árbol en el orden Izquierdo → Raíz → Derecho, lo que produce los registros ordenados ascendentemente por ID

Recorrido In-Orden (ordenado por ID):

```
[ID: 20 | Nombre: Bautismo Gregorio Gutiérrez | Año: 1801 | Tipo: Registro Civil]
[ID: 30 | Nombre: Carta Gobernación Antioquia | Año: 1813 | Tipo: Carta]
[ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]
[ID: 50 | Nombre: Acta Fundacional Medellín | Año: 1675 | Tipo: Acta]
[ID: 60 | Nombre: Defunción Juan de Dios Uribe | Año: 1900 | Tipo: Registro Civil]
[ID: 70 | Nombre: Escritura Hacienda El Retiro | Año: 1890 | Tipo: Escritura]
[ID: 80 | Nombre: Testamento Tomás Cipriano | Año: 1862 | Tipo: Testamento]
```

4.2 Recorrido Pre-Orden

El recorrido pre-orden sigue el orden Raíz → Izquierdo → Derecho. Útil para crear copias de seguridad que puedan reconstruir el árbol con la misma estructura.

Recorrido Pre-Orden:

```
[ID: 50 | Nombre: Acta Fundacional Medellín | Año: 1675 | Tipo: Acta]
[ID: 30 | Nombre: Carta Gobernación Antioquia | Año: 1813 | Tipo: Carta]
[ID: 20 | Nombre: Bautismo Gregorio Gutiérrez | Año: 1801 | Tipo: Registro Civil]
[ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]
[ID: 70 | Nombre: Escritura Hacienda El Retiro | Año: 1890 | Tipo: Escritura]
[ID: 60 | Nombre: Defunción Juan de Dios Uribe | Año: 1900 | Tipo: Registro Civil]
[ID: 80 | Nombre: Testamento Tomás Cipriano | Año: 1862 | Tipo: Testamento]
```

4.3 Recorrido Post-Orden

El recorrido post-orden sigue el orden Izquierdo → Derecho → Raíz. Es útil para procesos de eliminación jerárquica (se eliminan los hijos antes que el padre)

Recorrido Post-Orden:

```
[ID: 20 | Nombre: Bautismo Gregorio Gutiérrez | Año: 1801 | Tipo: Registro Civil]
[ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]
[ID: 30 | Nombre: Carta Gobernación Antioquia | Año: 1813 | Tipo: Carta]
[ID: 60 | Nombre: Defunción Juan de Dios Uribe | Año: 1900 | Tipo: Registro Civil]
[ID: 80 | Nombre: Testamento Tomás Cipriano | Año: 1862 | Tipo: Testamento]
[ID: 70 | Nombre: Escritura Hacienda El Retiro | Año: 1890 | Tipo: Escritura]
[ID: 50 | Nombre: Acta Fundacional Medellín | Año: 1675 | Tipo: Acta]
```

4.4 Búsqueda por ID

Se muestra la búsqueda de un ID existente (ID=40) y de uno inexistente (ID=99)

```
Seleccione una opción: 2
ID a buscar: 40
Encontrado: [ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]
```

```
Seleccione una opción: 2
ID a buscar: 1
No existe registro con ID 1
```

4.5 Estadísticas del árbol

```
Estadísticas del árbol:
Total de nodos : 7
Altura del árbol: 3
```

4.6 Eliminación de nodos

Se prueba la eliminación del nodo ID=30 (caso 3: dos hijos, sucesor=40) y luego del nodo ID=80 (caso 1: nodo hoja). En ambos casos el in-orden resultante confirma que el orden BST se preserva

```
Seleccione una opción: 3
ID a eliminar: 30
✓ Registro con ID 30 eliminado correctamente.
```

```
Recorrido In-Orden (ordenado por ID):
[ID: 20 | Nombre: Bautismo Gregorio Gutiérrez | Año: 1801 | Tipo: Registro Civil]
[ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]
[ID: 50 | Nombre: Acta Fundacional Medellín | Año: 1675 | Tipo: Acta]
[ID: 60 | Nombre: Defunción Juan de Dios Uribe | Año: 1900 | Tipo: Registro Civil]
[ID: 70 | Nombre: Escritura Hacienda El Retiro | Año: 1890 | Tipo: Escritura]
[ID: 80 | Nombre: Testamento Tomás Cipriano | Año: 1862 | Tipo: Testamento]
```

```
Seleccione una opción: 3
ID a eliminar: 80
✓ Registro con ID 80 eliminado correctamente.
```

Recorrido In-Orden (ordenado por ID):

[ID: 20 | Nombre: Bautismo Gregorio Gutiérrez | Año: 1801 | Tipo: Registro Civil]

[ID: 40 | Nombre: Matrimonio Familia Ospina | Año: 1856 | Tipo: Registro Civil]

[ID: 50 | Nombre: Acta Fundacional Medellín | Año: 1675 | Tipo: Acta]

[ID: 60 | Nombre: Defunción Juan de Dios Uribe | Año: 1900 | Tipo: Registro Civil]

[ID: 70 | Nombre: Escritura Hacienda El Retiro | Año: 1890 | Tipo: Escritura]